United States Patent	12405885
Kind Code	B2
Date of Patent	September 02, 2025
Inventor(s)	Dong; Yao Zu et al.

Avoidance of garbage collection in high performance memory management systems

Abstract

Systems, apparatuses and methods may provide for technology that detects a creation of a thread, dedicates a memory region to objects associated with the thread, and conducts a reclamation of the memory region in response to a termination of the thread. In one example, the memory region is a heap region and the reclamation bypasses at least a pause phase and a copy phase of a garbage collection process with respect to the heap region.

Inventors: **Dong; Yao Zu (Shanghai, CN), Shi; Qiming (Shanghai, CN), Xie; Chao (Shanghai, CN), Yang; Bin (Shanghai, CN), Zhou; Zhen (Shanghai, CN)**

Applicant: **INTEL CORPORATION** (Santa Clara, CA)

Family ID: **73552673**

Assignee: **INTEL CORPORATION (Santa Clara, CA)**

Appl. No.: **17/598190**

Filed (or PCT Filed): **May 31, 2019**

PCT No.: **PCT/CN2019/089503**

PCT Pub. No.: **WO2020/237621**

PCT Pub. Date: December 03, 2020

Prior Publication Data

Document Identifier	Publication Date
US 20220171704 A1	Jun. 02, 2022

Publication Classification

Int. Cl.: **G06F12/02** (20060101); **G06F9/50** (20060101); **G06F9/54** (20060101)

U.S. Cl.:

CPC **G06F12/0253** (20130101); **G06F9/5016** (20130101); **G06F9/5022** (20130101); **G06F9/544** (20130101); G06F2212/1016 (20130101); G06F2212/7205 (20130101)

Field of Classification Search

USPC: None

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
6823518	12/2003	Bliss	717/125	G06F 9/451
6842853	12/2004	Bush	712/228	G06F 9/461
7945911	12/2010	Garthwaite	707/814	G06F 9/4843
10019341	12/2017	Ma	N/A	N/A
2002/0049719	12/2001	Shiomi et al.	N/A	N/A
2004/0158589	12/2003	Liang et al.	N/A	N/A
2006/0074988	12/2005	Imanishi et al.	N/A	N/A
2006/0085433	12/2005	Bacon	N/A	G06F 12/0253
2007/0169042	12/2006	Janczewski	717/149	G06F 8/45
2008/0021939	12/2007	Dahlstedt	N/A	G06F 9/52
2009/0083509	12/2008	Johnson	711/170	G06F 12/0253
2011/0252216	12/2010	Ylonen	711/170	G06F 9/52

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
1761949	12/2005	CN	N/A
108073520	12/2017	CN	N/A
H04-142867	12/1991	JP	N/A
2002-259146	12/2001	JP	N/A
2004-078636	12/2003	JP	N/A
2009037546	12/2008	JP	N/A
2011134202	12/2010	JP	N/A
2009040228	12/2008	WO	N/A
2016097680	12/2015	WO	N/A
2017053109	12/2016	WO	N/A
2018152229	12/2017	WO	N/A

OTHER PUBLICATIONS

International Search Report and Written Opinion for International Patent Application No. PCT/CN2019/089503, mailed Feb. 28, 2020, 8 pages. cited by applicant

Ankit Bisht, “Stack vs Heap Memory Allocation”, <geeksforgeeks.org/stack-vs-heap-memory-allocation/>, retrieved May 15, 2019, 4 pages. cited by applicant

Ahmed Hussein et al., “Impact of GC Design on Power and Performance for Android”, SYSTOR, ACM, May 26, 2015, 12 pages, Halfa Israel. cited by applicant
Sangmin Lee, “Understanding Java Garbage Collection”, <cubrid.org/blog/understanding-java-garbage-collection>, May 31, 2017, 15 pages. cited by applicant
F. Pizlo et al., “Real-Time Java Scoped Memory: Design Patterns and Semantics”, Seventh IEEE International Symposium on Object-Oriented Real-Time Distributed Computing, May 2004, 10 pages, Vienna, Austria. cited by applicant
International Searching Authority, “International Preliminary Report on Patentability,” issued in connection with International Patent Application No. PCT/CN2019/089503, issued on Nov. 16, 2021, 4 pages. cited by applicant
European Patent Office, “Extended European Search Report,” issued in connection with European Patent Application No. 19931226.5, dated Dec. 7, 2022, 9 pages. cited by applicant
European Patent Office, “Communication Under Rule 71(3) EPC,” issued in connection with European Patent Application No. 19931226.5, dated Apr. 17, 2024, 6 pages. cited by applicant
European Patent Office, “Decision to Grant a European Patent Pursuant to Article 97(1) EPC,” issued in connection with European Patent Application No. 19931226.5, dated Aug. 16, 2024, 2 pages. cited by applicant
Japan Patent Office, “Search Report by Registered Search Organization,” issued in connection with Japanese Patent Application No. 2021-563430, dated May 18, 2023, 40 pages. [With English Translation]. cited by applicant
Japan Patent Office, “Notice of Reasons for Refusal,” issued in connection with Japanese Patent Application No. 2021-563430, dated Jun. 20, 2023, 6 pages. [With English Translation]. cited by applicant
Japan Patent Office, “Notice of Reasons for Refusal,” issued in connection with Japanese Patent Application No. 2021-563430 dated Dec. 12, 2023, 6 pages. [With English Translation]. cited by applicant
Japan Patent Office, “Decision of Refusal,” issued in connection with Japanese Patent Application No. 2021-563430, dated May 28, 2024, 4 pages. [With English Translation]. cited by applicant
Japan Patent Office, “Decision of Dismissal of Amendment,” issued in connection with Japanese Patent Application No. 2021-563430, dated May 28, 2024, 5 pages. [With English Translation]. cited by applicant
Japan Patent Office, “Reconsideration Report by Examiner before Appeal,” issued in connection with Japanese Patent Application No. 2021-563430, dated Jan. 15, 2025, 4 pages. [With English Translation]. cited by applicant
Japan Patent Office, “Notice of Reasons for Refusal,” issued in connection with Japanese Patent Application No. 2021-563430, dated Jul. 10, 2025, 7 pages. [With English Translation]. cited by applicant

---

Primary Examiner: Perez-Velez; Rocio Del Mar

Assistant Examiner: Ayash; Marwan

Attorney, Agent or Firm: Hanley, Flight & Zimmerman, LLC

Background/Summary

CROSS REFERENCE TO RELATED APPLICATIONS
(1) This application is a U.S. National Phase Patent Application which claims benefit to

International Patent Application No. PCT/CN2019/089503 filed on May 31, 2019.
TECHNICAL FIELD
(2) Embodiments generally relate to memory management in computing architectures. More particularly, embodiments relate to the avoidance of garbage collection in high performance memory management systems.
BACKGROUND
(3) During operation, computer programs may allocate memory to various objects in order to perform operations involved in executing the programs. When computer programs fail to de-allocate memory from objects that are no longer used, available memory reduces unnecessarily, which may have a negative impact on performance and may lead to system failures (e.g., when no additional memory is available). While programs such as managed runtime applications may use "garbage collectors" to automatically reclaim memory that is no longer being used, there remains considerable room for improvement. For example, traditional garbage collectors may have a negative impact on performance in terms of processor usage, unpredicted pauses in user interfaces, additional power consumption (e.g., reduced battery life), and so forth.

Description

BRIEF DESCRIPTION OF THE DRAWINGS
(1) The various advantages of the embodiments will become apparent to one skilled in the art by reading the following specification and appended claims, and by referencing the following drawings, in which:
(2) FIG. **1** is an illustration of an example of an allocation space after threads are created according to an embodiment;
(3) FIG. **2** is a flow diagram of an example of a dedication of a memory region to objects associated with a thread according to an embodiment;
(4) FIG. **3** is an illustration of an example of an allocation space after a memory reclamation according to an embodiment;
(5) FIG. **4** is a flow diagram of an example of a memory reclamation according to an embodiment;
(6) FIGS. **5** and **6** are flowcharts of examples of methods of managing memory according to an embodiment;
(7) FIG. **7** is a block diagram of an example of a performance-enhanced computing system according to an embodiment;
(8) FIG. **8** is an illustration of an example of a semiconductor package apparatus according to an embodiment;
(9) FIG. **9** is a block diagram of an example of a processor according to an embodiment; and
(10) FIG. **10** is a block diagram of an example of a multi-processor based computing system according to an embodiment.
DESCRIPTION OF EMBODIMENTS
(11) Turning now to FIG. **1**, an allocation space **20** (**20***a*-**20***e*) is shown after threads are created. In an embodiment, the allocation space **20** is part of a heap memory that is used to conduct dynamic memory allocation during run-time operation of a computer program that creates and terminates (e.g., destroys) threads. The heap memory may also include other memory space (not shown) such as, for example, large object space (e.g., storing relatively large primitive arrays and/or string arrays), image space (e.g., storing executable files, resource files, etc.), shared space (e.g., storing objects shared by multiple processes), non-moving space (e.g., storing long-lived objects), and so forth. In one example, the allocation space **20** stores relatively short-lived objects that might be targeted during a garbage collection process. Moreover, the threads may correspond to user interface activities that provide rich special effects and cause the short-lived objects to consume a

relatively large amount of memory.

(12) In the illustrated example, the allocation space **20** includes a first region **20***a* ("Region **1**"), a second region **20***b* ("Region **2**"), a third region **20***c* ("Region **3**"), a fourth region **20***d* ("Region **4**"), and a fifth region **20***e* ("Region **5**"). The number of regions shown is for discussion purposes only and may vary depending on the circumstances. Regions of the illustrated allocation space **20** are individually dedicated to specific threads and/or activities. For example, the second region **20***b* and the third region **20***c* may be dedicated to a first thread ("Thread **1**"), the fourth region **20***d* might be dedicated to a second thread ("Thread **2**"), and so forth. In an embodiment, the objects in the second region **20***b* and the third region **20***c* may be shared between functions associated with the first thread without conducting extra copy operations. Similarly, the objects in the fourth region **20***d* may be shared between functions associated with the second thread without conducting extra copy operations. As will be discussed in greater detail, the illustrated solution enables performance to be enhanced.

(13) FIG. **2** shows a flow diagram **22** in which a memory allocator (e.g., framework layer runtime manager) detects a function **24** (callback method "onCreate( )") that will create a new thread. In the illustrated example, the memory allocator triggers the dedication of a memory region to the thread by issuing a first command **26** ("StartMemoryRegion( )"). When the application calls the function **24**, all objects (e.g., "Object a", "Object b") associated with the new thread will be allocated to the dedicated memory region. In addition to calling the function **24**, the memory allocator may set a flag in the calling thread object to activate thread based allocation. In an embodiment, the memory allocator also maps the thread to the dedicated memory region. Table I below shows an example of such a mapping.

(14) TABLE-US-00001 TABLE I UI Thread ID Region 3 Region 2, Region 3 4 Region 4

(15) When the function **24** is done allocating objects, the illustrated memory allocator issues a second command **28** ("StopMemoryRegion( )") to discontinue dedicating objects to the memory region in question. The memory allocator may also clear the flag in the calling thread object to deactivate the thread based allocation in response to the termination of the thread. In an embodiment, the memory allocator also clears the appropriate thread-to-region table mapping entries. An example of pseudocode to implement the first command **26** and the second command **28** is shown below.

(16) TABLE-US-00002 StartMemoryRegion( ) { threadId = GetThreadId( ); SetActivityModeAllocate(threadId); regionId = AllocateNewRegion( ); Record in the table with threadId and regionId; } StopMemoryRegion( ) { threadId = GetThreadId( ); UnsetActivityModeAllocate(threadId); }

(17) If the size of the selected region is insufficient, an additional unused region may be dedicated to the objects associated with the thread. In an embodiment, the memory allocator checks whether the thread is under thread allocation mode. If not, the memory allocator may switch to a normal allocation mode. Otherwise, the memory allocator searches the table to find the specific region identifier and attempts to allocate the object in the designated region. If the region is full, the memory allocator may allocate another region for the thread and update the table correspondingly. An example of pseudocode to implement the selection of an additional region is shown below.

(18) TABLE-US-00003 AllocateObject( ) { threadId = GetThreadId( ); checkActivityMode(threadId); If not true: //Origin code path to allocate objects else: regionid = FindRegionID(threadid); AllocateObject(regionid); If not enough space: newRegionId = AllocateNewRegion( ); Update the newRegionId to the table AllocateObject(newRegionid); }

(19) With continuing reference to FIGS. **1** and **3**, termination of a thread may trigger a reclamation of all regions dedicated to the thread. For example, the termination of Thread **1** results in all objects in the second region **20***b* and the third region **20***c* being reclaimed. In an embodiment, the termination of the thread is detected based on high level (e.g., upper layer) information from the

application. Of particular note is that garbage collector (GC) operations such as a pause phase in which all other threads are paused (e.g., to permit the GC to collect a root object set and determine which regions are to be reclaimed), may be bypassed. Other GC operations such as a copy phase in which the GC scans all living objects starting from the root set and copies those objects to an unused region (e.g., destination region), are also bypassed in the illustrated solution. Indeed, if the thread produces only short-lived objects, the frequency of executing GC operations may approach zero. Accordingly, performance is enhanced in terms of processor usage, unpredicted pauses in user interfaces, additional power consumption (e.g., reduced battery life), and so forth.

(20) FIG. **4** shows a flow diagram **33** in which the memory allocator detects a function **32** (callback method "onDestroy( )") that will terminate an existing thread. In an embodiment, the function **32** results from too many applications running in the background, the application being closed and removed from an active application list, and so forth. In the illustrated example, the memory allocator triggers the reclamation of one or more memory regions that are dedicated to the thread by issuing a command **34** ("ReclaimMemoryRegion( )"). When the application calls the function **32**, all memory regions that are dedicated to the thread will be reclaimed. In an embodiment, the memory allocator locates the selected regions for the thread by searching a table such as, for example, Table I for the appropriate thread identifier. The regions may be reclaimed one at a time. An example of pseudocode to implement the command **34** is shown below.

(21) TABLE-US-00004 ReclaimMemoryRegion( ) { threadid = GetThreadId( ); allRegionIds = FindAllRegionIDs(threadid); For each id in allRegionIds: FreeRegion(id); }

(22) FIG. **5** shows a method **40** of managing memory. The method **40** may generally be implemented in a memory allocator such as, for example, the aforementioned memory allocator (FIG. **1**). More particularly, the method **40** may be implemented as one or more modules in a set of logic instructions stored in a machine- or computer-readable storage medium such as random access memory (RAM), read only memory (ROM), programmable ROM (PROM), firmware, flash memory, etc., in configurable logic such as, for example, programmable logic arrays (PLAs), field programmable gate arrays (FPGAs), complex programmable logic devices (CPLDs), in fixed-functionality hardware logic using circuit technology such as, for example, application specific integrated circuit (ASIC), complementary metal oxide semiconductor (CMOS) or transistor-transistor logic (TTL) technology, or any combination thereof.

(23) For example, computer program code to carry out operations shown in the method **40** may be written in any combination of one or more programming languages, including an object oriented programming language such as JAVA, SMALLTALK, C++ or the like and conventional procedural programming languages, such as the "C" programming language or similar programming languages. Additionally, logic instructions might include assembler instructions, instruction set architecture (ISA) instructions, machine instructions, machine dependent instructions, microcode, state-setting data, configuration data for integrated circuitry, state information that personalizes electronic circuitry and/or other structural components that are native to hardware (e.g., host processor, central processing unit/CPU, microcontroller, etc.).

(24) Illustrated processing block **42** detects the initiation of a thread, where a first memory region is dedicated to objects associated with the thread at block **44**. In an embodiment, block **44** includes activating thread based allocation and mapping the thread to the first memory region. A determination may be made at block **46** as to whether a termination of the thread has been initiated. If not, the illustrated method **40** repeats block **46**. Of particular note is that the objects associated with the same thread are expected to have a similar lifecycle. Accordingly, upon detection of the thread termination, block **48** conducts a first reclamation of the first memory region in response to termination of the thread. In an embodiment, the method **40** further includes deactivating thread based allocation in response to the termination of the thread and unmapping the thread from the first memory region. In one example, the first memory region is a heap region and the first

reclamation bypasses a pause phase and a copy phase of a garbage collection process with respect to the heap region. Additionally, the thread may correspond to a user interface (UI) activity. The method **40** may therefore enhance performance in terms of processor usage, unpredicted pauses in user interfaces, additional power consumption (e.g., reduced battery life), and so forth.

(25) The illustrated method **40** may be used for native applications such as, for example, C or C++ applications, as well as for managed runtime applications having garbage collectors, which automatically reclaim memory that is no longer being used. The managed runtime applications may include, but are not limited to, for example, HTML5 (Hypertext Markup Language 5, e.g., HTML 5.2, W3C Recommendation, 14 Dec. 2017), JAVASCRIPT, C # (e.g., C #7.3, MICROSOFT Corp., May 7, 2018), Ruby (e.g., Ruby 2.6.3, Y. Matsumoto, Apr. 17, 2019), Perl (e.g., Perl 5.28.2, Perl.org, Apr. 19, 2019), Python (e.g., Python 3.7.3, Python Software Foundation, Mar. 25, 2019), JAVA (e.g., JAVA 10, ORACLE Corp., Mar. 20, 2018), etc.

(26) FIG. **6** shows another method **50** of managing memory. The method **50** may generally be implemented in a memory allocator such as, for example, the aforementioned memory allocator (FIG. **1**). More particularly, the method **50** may be implemented as one or more modules in a set of logic instructions stored in a machine- or computer-readable storage medium such as RAM, ROM, PROM, firmware, flash memory, etc., in configurable logic such as, for example, PLAs, FPGAs, CPLDs, in fixed-functionality hardware logic using circuit technology such as, for example, ASIC, CMOS or TTL technology, or any combination thereof.

(27) Illustrated processing block **52** determines whether the first memory region is full. If so, a second memory region may be dedicated at block **54** to the objects associated with the thread. Additionally, a determination may be made at block **56** as to whether a termination of the thread has been initiated. If not, the illustrated method **50** repeats block **56**. Upon detection of the thread termination, block **58** conducts a second reclamation of the second memory region in response to termination of the thread. Accordingly, the method **50** further enhances performance by supporting threads that allocate a relatively large number of objects (e.g., much larger than a stack, and therefore eliminating concerns over a stack overflow).

(28) Turning now to FIG. **7**, a performance-enhanced computing system **150** is shown. The system **150** may generally be part of an electronic device/platform having computing functionality (e.g., personal digital assistant/PDA, notebook computer, tablet computer, convertible tablet, server), communications functionality (e.g., smart phone), imaging functionality (e.g., camera, camcorder), media playing functionality (e.g., smart television/TV), wearable functionality (e.g., watch, eyewear, headwear, footwear, jewelry), vehicular functionality (e.g., car, truck, motorcycle), robotic functionality (e.g., autonomous robot), etc., or any combination thereof. In the illustrated example, the system **150** includes a host processor **152** (e.g., central processing unit/CPU) having an integrated memory controller (IMC) **154** that is coupled to a system memory **156**.

(29) The illustrated system **150** also includes an input output (IO) module **158** implemented together with the host processor **152** and a graphics processor **160** on a semiconductor die **162** as a system on chip (SoC). The illustrated IO module **158** communicates with, for example, a display **164** (e.g., touch screen, liquid crystal display/LCD, light emitting diode/LED display), a network controller **166** (e.g., wired and/or wireless NIC), and mass storage **168** (e.g., hard disk drive/HDD, optical disk, solid state drive/SSD, flash memory).

(30) In an embodiment, the host processor **152** and/or the IO module **158** execute program instructions **170** retrieved from the system memory **156** and/or the mass storage **168** to perform one or more aspects of the method **40** (FIG. **5**) and/or the method **50** (FIG. **6**), already discussed. Thus, execution of the illustrated instructions **170** may cause the computing system **150** to detect a creation of a thread, dedicate a first memory region in the system memory **156** to objects associated with the thread, and conduct a first reclamation of the first memory region in response to a termination of the thread. In an embodiment, the first memory region is a heap region and the first reclamation bypasses at least a pause phase and a copy phase of a garbage collection process with

respect to the heap region. Additionally, the thread may correspond to a UI activity (e.g., of a high performance game application) that causes the objects to be short-lived and to consume a relatively large amount of the system memory **156**. In such a case the illustrated display **164** visually presents information associated with the UI activity.

(31) As already noted, GC operations such as a pause phase in which all other threads are paused (e.g., to permit the GC to collect the root object set and determine which regions are to be reclaimed), may be bypassed. Other GC operations such as a copy phase in which the GC scans all living objects starting from the root set and copies those objects to an unused region (e.g., destination region), are also bypassed in the illustrated solution. Accordingly, the performance of the computing system **150** is enhanced in terms of processor usage, unpredicted pauses in user interfaces, additional power consumption (e.g., reduced battery life), and so forth.

(32) FIG. **8** shows a semiconductor apparatus **172** (e.g., chip, die, package). The illustrated apparatus **172** includes one or more substrates **174** (e.g., silicon, sapphire, gallium arsenide) and logic **176** (e.g., transistor array and other integrated circuit/IC components) coupled to the substrate(s) **174**. In an embodiment, the logic **176** implements one or more aspects of method **40** (FIG. **5**) and/or the method **50** (FIG. **6**), already discussed. Thus, the logic **176** may detect a creation of a thread, dedicate a first memory region to objects associated with the thread, and conduct a first reclamation of the first memory region in response to a termination of the thread. In an embodiment, the first memory region is a heap region and the first reclamation bypasses (e.g., avoids) at least a pause phase and a copy phase of a garbage collection process with respect to the heap region. The performance of the apparatus **172** is therefore enhanced in terms of processor usage, unpredicted pauses in user interfaces, additional power consumption (e.g., reduced battery life), and so forth.

(33) The logic **176** may be implemented at least partly in configurable logic or fixed-functionality hardware logic. In one example, the logic **176** includes transistor channel regions that are positioned (e.g., embedded) within the substrate(s) **174**. Thus, the interface between the logic **176** and the substrate(s) **174** may not be an abrupt junction. The logic **176** may also be considered to include an epitaxial layer that is grown on an initial wafer of the substrate(s) **174**.

(34) FIG. **9** illustrates a processor core **200** according to one embodiment. The processor core **200** may be the core for any type of processor, such as a micro-processor, an embedded processor, a digital signal processor (DSP), a network processor, or other device to execute code. Although only one processor core **200** is illustrated in FIG. **9**, a processing element may alternatively include more than one of the processor core **200** illustrated in FIG. **9**. The processor core **200** may be a single-threaded core or, for at least one embodiment, the processor core **200** may be multithreaded in that it may include more than one hardware thread context (or "logical processor") per core.

(35) FIG. **9** also illustrates a memory **270** coupled to the processor core **200**. The memory **270** may be any of a wide variety of memories (including various layers of memory hierarchy) as are known or otherwise available to those of skill in the art. The memory **270** may include one or more code **213** instruction(s) to be executed by the processor core **200**, wherein the code **213** may implement method **40** (FIG. **5**) and/or the method **50** (FIG. **6**), already discussed. The processor core **200** follows a program sequence of instructions indicated by the code **213**. Each instruction may enter a front end portion **210** and be processed by one or more decoders **220**. The decoder **220** may generate as its output a micro operation such as a fixed width micro operation in a predefined format, or may generate other instructions, microinstructions, or control signals which reflect the original code instruction. The illustrated front end portion **210** also includes register renaming logic **225** and scheduling logic **230**, which generally allocate resources and queue the operation corresponding to the convert instruction for execution.

(36) The processor core **200** is shown including execution logic **250** having a set of execution units **255-1** through **255**-N. Some embodiments may include a number of execution units dedicated to specific functions or sets of functions. Other embodiments may include only one execution unit or

one execution unit that can perform a particular function. The illustrated execution logic **250** performs the operations specified by code instructions.

(37) After completion of execution of the operations specified by the code instructions, back end logic **260** retires the instructions of the code **213**. In one embodiment, the processor core **200** allows out of order execution but requires in order retirement of instructions. Retirement logic **265** may take a variety of forms as known to those of skill in the art (e.g., re-order buffers or the like). In this manner, the processor core **200** is transformed during execution of the code **213**, at least in terms of the output generated by the decoder, the hardware registers and tables utilized by the register renaming logic **225**, and any registers (not shown) modified by the execution logic **250**.

(38) Although not illustrated in FIG. **9**, a processing element may include other elements on chip with the processor core **200**. For example, a processing element may include memory control logic along with the processor core **200**. The processing element may include I/O control logic and/or may include I/O control logic integrated with memory control logic. The processing element may also include one or more caches.

(39) Referring now to FIG. **10**, shown is a block diagram of a computing system **1000** embodiment in accordance with an embodiment. Shown in FIG. **10** is a multiprocessor system **1000** that includes a first processing element **1070** and a second processing element **1080**. While two processing elements **1070** and **1080** are shown, it is to be understood that an embodiment of the system **1000** may also include only one such processing element.

(40) The system **1000** is illustrated as a point-to-point interconnect system, wherein the first processing element **1070** and the second processing element **1080** are coupled via a point-to-point interconnect **1050**. It should be understood that any or all of the interconnects illustrated in FIG. **10** may be implemented as a multi-drop bus rather than point-to-point interconnect.

(41) As shown in FIG. **10**, each of processing elements **1070** and **1080** may be multicore processors, including first and second processor cores (i.e., processor cores **1074***a* and **1074***b* and processor cores **1084***a* and **1084***b*). Such cores **1074***a*, **1074***b*, **1084***a*, **1084***b* may be configured to execute instruction code in a manner similar to that discussed above in connection with FIG. **9**.

(42) Each processing element **1070**, **1080** may include at least one shared cache **1896***a*, **1896***b*. The shared cache **1896***a*, **1896***b* may store data (e.g., instructions) that are utilized by one or more components of the processor, such as the cores **1074***a*, **1074***b* and **1084***a*, **1084***b*, respectively. For example, the shared cache **1896***a*, **1896***b* may locally cache data stored in a memory **1032**, **1034** for faster access by components of the processor. In one or more embodiments, the shared cache **1896***a*, **1896***b* may include one or more mid-level caches, such as level 2 (L2), level 3 (L3), level 4 (L4), or other levels of cache, a last level cache (LLC), and/or combinations thereof.

(43) While shown with only two processing elements **1070**, **1080**, it is to be understood that the scope of the embodiments are not so limited. In other embodiments, one or more additional processing elements may be present in a given processor. Alternatively, one or more of processing elements **1070**, **1080** may be an element other than a processor, such as an accelerator or a field programmable gate array. For example, additional processing element(s) may include additional processors(s) that are the same as a first processor **1070**, additional processor(s) that are heterogeneous or asymmetric to processor a first processor **1070**, accelerators (such as, e.g., graphics accelerators or digital signal processing (DSP) units), field programmable gate arrays, or any other processing element. There can be a variety of differences between the processing elements **1070**, **1080** in terms of a spectrum of metrics of merit including architectural, micro architectural, thermal, power consumption characteristics, and the like. These differences may effectively manifest themselves as asymmetry and heterogeneity amongst the processing elements **1070**, **1080**. For at least one embodiment, the various processing elements **1070**, **1080** may reside in the same die package.

(44) The first processing element **1070** may further include memory controller logic (MC) **1072** and point-to-point (P-P) interfaces **1076** and **1078**. Similarly, the second processing element **1080**

may include a MC **1082** and P-P interfaces **1086** and **1088**. As shown in FIG. **10**, MC's **1072** and **1082** couple the processors to respective memories, namely a memory **1032** and a memory **1034**, which may be portions of main memory locally attached to the respective processors. While the MC **1072** and **1082** is illustrated as integrated into the processing elements **1070**, **1080**, for alternative embodiments the MC logic may be discrete logic outside the processing elements **1070**, **1080** rather than integrated therein.

(45) The first processing element **1070** and the second processing element **1080** may be coupled to an I/O subsystem **1090** via P-P interconnects **1076 1086**, respectively. As shown in FIG. **10**, the I/O subsystem **1090** includes P-P interfaces **1094** and **1098**. Furthermore, I/O subsystem **1090** includes an interface **1092** to couple I/O subsystem **1090** with a high performance graphics engine **1038**. In one embodiment, bus **1049** may be used to couple the graphics engine **1038** to the I/O subsystem **1090**. Alternately, a point-to-point interconnect may couple these components.

(46) In turn, I/O subsystem **1090** may be coupled to a first bus **1016** via an interface **1096**. In one embodiment, the first bus **1016** may be a Peripheral Component Interconnect (PCI) bus, or a bus such as a PCI Express bus or another third generation I/O interconnect bus, although the scope of the embodiments are not so limited.

(47) As shown in FIG. **10**, various I/O devices **1014** (e.g., biometric scanners, speakers, cameras, sensors) may be coupled to the first bus **1016**, along with a bus bridge **1018** which may couple the first bus **1016** to a second bus **1020**. In one embodiment, the second bus **1020** may be a low pin count (LPC) bus. Various devices may be coupled to the second bus **1020** including, for example, a keyboard/mouse **1012**, communication device(s) **1026**, and a data storage unit **1019** such as a disk drive or other mass storage device which may include code **1030**, in one embodiment. The illustrated code **1030** may implement the method **40** (FIG. **5**) and/or the method **50** (FIG. **6**), already discussed, and may be similar to the code **213** (FIG. **9**), already discussed. Further, an audio I/O **1024** may be coupled to second bus **1020** and a battery **1010** may supply power to the computing system **1000**.

(48) Note that other embodiments are contemplated. For example, instead of the point-to-point architecture of FIG. **10**, a system may implement a multi-drop bus or another such communication topology. Also, the elements of FIG. **10** may alternatively be partitioned using more or fewer integrated chips than shown in FIG. **10**.

ADDITIONAL NOTES AND EXAMPLES

(49) Example 1 includes a performance-enhanced computing system comprising a display, a processor coupled to the display, and a memory coupled to the processor, the memory including a set of executable program instructions, which when executed by the processor, cause the computing system to detect a creation of a thread, dedicate a first memory region to objects associated with the thread, and conduct a first reclamation of the first memory region in response to a termination of the thread.

(50) Example 2 includes the computing system of Example 1, wherein the first memory region is a heap region and the first reclamation is to bypass a pause phase and a copy phase of a garbage collection process with respect to the heap region.

(51) Example 3 includes the computing system of Example 1, wherein to dedicate the first memory region to the objects associated with the thread, the executable program instructions, when executed, cause the computing system to activate thread based allocation, and map the thread to the first memory region.

(52) Example 4 includes the computing system of Example 3, wherein the executable program instructions, when executed, cause the computing system to deactivate the thread based allocation in response to the termination of the thread, and unmap the thread from the first memory region.

(53) Example 5 includes the computing system of Example 1, wherein the executable program instructions, when executed, cause the computing system to dedicate a second memory region to the objects associated with the thread in response to a determination that the first memory region is

full, and conduct a second reclamation of the second memory region in response to the termination of the thread.
(54) Example 6 includes the computing system of any one of Examples 1 to 5, wherein the thread is to correspond to a user interface activity, and wherein the display is to visually present information associated with the user interface activity.
(55) Example 7 includes a semiconductor apparatus comprising one or more substrates, and logic coupled to the one or more substrates, wherein the logic is implemented at least partly in one or more of configurable logic or fixed-functionality hardware logic, the logic coupled to the one or more substrates to detect a creation of a thread, dedicate a first memory region to objects associated with the thread, and conduct a first reclamation of the first memory region in response to a termination of the thread.
(56) Example 8 includes the apparatus of Example 7, wherein the first memory region is a heap region and the first reclamation is to bypass a pause phase and a copy phase of a garbage collection process with respect to the heap region.
(57) Example 9 includes the apparatus of Example 7, wherein to dedicate the first memory region to the objects associated with the thread, the logic coupled to the one or more substrates is to activate thread based allocation, and map the thread to the first memory region.
(58) Example 10 includes the apparatus of Example 9, wherein the logic coupled to the one or more substrates is to deactivate the thread based allocation in response to the termination of the thread, and unmap the thread from the first memory region.
(59) Example 11 includes the apparatus of Example 7, wherein the logic coupled to the one or more substrates is to dedicate a second memory region to the objects associated with the thread in response to a determination that the first memory region is full, and conduct a second reclamation of the second memory region in response to the termination of the thread.
(60) Example 12 includes the apparatus of any one of Examples 7 to 11, wherein the thread is to correspond to a user interface activity.
(61) Example 13 includes at least one computer readable storage medium comprising a set of executable program instructions, which when executed by a computing system, cause the computing system to detect a creation of a thread, dedicate a first memory region to objects associated with the thread, and conduct a first reclamation of the first memory region in response to a termination of the thread.
(62) Example 14 includes the at least one computer readable storage medium of Example 13, wherein the first memory region is a heap region and the first reclamation is to bypass a pause phase and a copy phase of a garbage collection process with respect to the heap region.
(63) Example 15 includes the at least one computer readable storage medium of Example 13, wherein to dedicate the first memory region to the objects associated with the thread, the executable program instructions, when executed, cause the computing system to activate thread based allocation, and map the thread to the first memory region.
(64) Example 16 includes the at least one computer readable storage medium of Example 15, wherein the executable program instructions, when executed, cause the computing system to deactivate the thread based allocation in response to the termination of the thread, and unmap the thread from the first memory region.
(65) Example 17 includes the at least one computer readable storage medium of Example 13, wherein the executable program instructions, when executed, cause the computing system to dedicate a second memory region to the objects associated with the thread in response to a determination that the first memory region is full, and conduct a second reclamation of the second memory region in response to the termination of the thread.
(66) Example 18 includes the at least one computer readable storage medium of any one of Examples 13 to 17, wherein the thread is to correspond to a user interface activity.
(67) Example 19 includes a method comprising detecting a creation of a thread, dedicating a first

memory region to objects associated with the thread, and conducting a first reclamation of the first memory region in response to a termination of the thread.
(68) Example 20 includes the method of Example 19, wherein the first memory region is a heap region and the first reclamation bypasses a pause phase and a copy phase of a garbage collection process with respect to the heap region.
(69) Example 21 includes the method of Example 19, wherein dedicating the first memory region to the objects associated with the thread includes activating thread based allocation, and mapping the thread to the first memory region.
(70) Example 22 includes the method of Example 21, further including deactivating the thread based allocation in response to the termination of the thread, and unmapping the thread from the first memory region.
(71) Example 23 includes the method of Example 19, further including dedicating a second memory region to the objects associated with the thread in response to a determination that the first memory region is full, and conducting a second reclamation of the second memory region in response to the termination of the thread.
(72) Example 24 includes the method of any one of Examples 19 to 23, wherein the thread corresponds to a user interface activity.
(73) Thus, technology described herein packs an activity and the objects created by the activity into a single object bundle and reclaims memory consumed by the objects all at once (e.g., and without invoking a GC thread). The technology may apply to any language and may focus on the reclamation of all objects sharing a similar lifetime. Moreover, the technology may not place any additional burden on application developers (e.g., all modifications may be made in the managed runtime framework layer). The technology is also transparent to users (e.g., no negative impact on the user experience) and supports the sharing of objects between functions without extra copying operations (e.g., as in threaded memory allocators and/or resource allocation is initialization/RAII solutions).
(74) Embodiments are applicable for use with all types of semiconductor integrated circuit (“IC”) chips. Examples of these IC chips include but are not limited to processors, controllers, chipset components, programmable logic arrays (PLAs), memory chips, network chips, systems on chip (SoCs), SSD/NAND controller ASICs, and the like. In addition, in some of the drawings, signal conductor lines are represented with lines. Some may be different, to indicate more constituent signal paths, have a number label, to indicate a number of constituent signal paths, and/or have arrows at one or more ends, to indicate primary information flow direction. This, however, should not be construed in a limiting manner. Rather, such added detail may be used in connection with one or more exemplary embodiments to facilitate easier understanding of a circuit. Any represented signal lines, whether or not having additional information, may actually comprise one or more signals that may travel in multiple directions and may be implemented with any suitable type of signal scheme, e.g., digital or analog lines implemented with differential pairs, optical fiber lines, and/or single-ended lines.
(75) Example sizes/models/values/ranges may have been given, although embodiments are not limited to the same. As manufacturing techniques (e.g., photolithography) mature over time, it is expected that devices of smaller size could be manufactured. In addition, well known power/ground connections to IC chips and other components may or may not be shown within the figures, for simplicity of illustration and discussion, and so as not to obscure certain aspects of the embodiments. Further, arrangements may be shown in block diagram form in order to avoid obscuring embodiments, and also in view of the fact that specifics with respect to implementation of such block diagram arrangements are highly dependent upon the platform within which the embodiment is to be implemented, i.e., such specifics should be well within purview of one skilled in the art. Where specific details (e.g., circuits) are set forth in order to describe example embodiments, it should be apparent to one skilled in the art that embodiments can be practiced

without, or with variation of, these specific details. The description is thus to be regarded as illustrative instead of limiting.
(76) The term “coupled” may be used herein to refer to any type of relationship, direct or indirect, between the components in question, and may apply to electrical, mechanical, fluid, optical, electromagnetic, electromechanical or other connections. In addition, the terms “first”, “second”, etc. may be used herein only to facilitate discussion, and carry no particular temporal or chronological significance unless otherwise indicated.
(77) As used in this application and in the claims, a list of items joined by the term “one or more of” may mean any combination of the listed terms. For example, the phrases “one or more of A, B or C” may mean A, B, C; A and B; A and C; B and C; or A, B and C.
(78) Those skilled in the art will appreciate from the foregoing description that the broad techniques of the embodiments can be implemented in a variety of forms. Therefore, while the embodiments have been described in connection with particular examples thereof, the true scope of the embodiments should not be so limited since other modifications will become apparent to the skilled practitioner upon a study of the drawings, specification, and following claims.

Claims

1. A performance-enhanced computing system comprising: a display; a processor coupled to the display; and a memory coupled to the processor, the memory including a set of executable program instructions, which when executed by the processor, cause the computing system to: detect a creation of a first thread, activate thread based allocation for the first thread based on the first thread being associated with user interface activity, switch to a normal allocation mode from the thread based allocation for a third thread based on the third thread not being associated with the user interface activity, dedicate a first memory region to objects associated with only the first thread to bypass a storage of objects associated with a second thread into the first memory region based on the thread based allocation being activated for the first thread, dedicate a second memory region to the objects associated with the second thread, execute the normal allocation mode for objects associated with the third thread to store the objects of the third thread in a shared space based on the third thread being switched to the normal allocation mode, and conduct a first reclamation of the first memory region in response to a termination of the first thread.
2. The computing system of claim 1, wherein the first memory region is a heap region and the first reclamation is to bypass a pause phase and a copy phase of a garbage collection process with respect to the heap region.
3. The computing system of claim 1, wherein to dedicate the first memory region to the objects associated with the first thread, the executable program instructions, when executed, cause the computing system to: map the first thread to the first memory region.
4. The computing system of claim 3, wherein the executable program instructions, when executed, cause the computing system to: deactivate the thread based allocation in response to the termination of the first thread; and unmap the first thread from the first memory region.
5. The computing system of claim 1, wherein the executable program instructions, when executed, cause the computing system to: dedicate a third memory region to the objects associated with the first thread in response to a determination that the first memory region is full; and conduct a second reclamation of the third memory region in response to the termination of the first thread.
6. The computing system of claim 1, wherein the first and second threads are to correspond to the user interface activity, and wherein the display is to visually present information associated with the user interface activity.
7. A semiconductor apparatus comprising: one or more substrates; and logic coupled to the one or more substrates, wherein the logic is implemented at least partly in one or more of configurable logic or fixed-functionality hardware logic, the logic coupled to the one or more substrates to:

detect a creation of a first thread; activate thread based allocation for the first thread based on the first thread being associated with user interface activity; switch to a normal allocation mode from the thread based allocation for a third thread based on the third thread not being associated with the user interface activity; dedicate a first memory region to objects associated with only the first thread to bypass a storage of objects associated with a second thread into the first memory region based on the thread based allocation being activated for the first thread; dedicate a second memory region to the objects associated with the second thread; execute the normal allocation mode for objects associated with the third thread to store the objects of the third thread in a shared space based on the third thread being switched to the normal allocation mode; and conduct a first reclamation of the first memory region in response to a termination of the first thread.

8. The apparatus of claim 7, wherein the first memory region is a heap region and the first reclamation is to bypass a pause phase and a copy phase of a garbage collection process with respect to the heap region.

9. The apparatus of claim 7, wherein to dedicate the first memory region to the objects associated with the first thread, the logic coupled to the one or more substrates is to: map the first thread to the first memory region.

10. The apparatus of claim 9, wherein the logic coupled to the one or more substrates is to: deactivate the thread based allocation in response to the termination of the first thread; and unmap the first thread from the first memory region.

11. The apparatus of claim 7, wherein the logic coupled to the one or more substrates is to: dedicate a third memory region to the objects associated with the first thread in response to a determination that the first memory region is full; and conduct a second reclamation of the third memory region in response to the termination of the first thread.

12. The apparatus of claim 7, wherein the first and second threads are to correspond to the user interface activity.

13. At least one non-transitory computer readable storage medium comprising a set of executable program instructions, which when executed by a computing system, cause the computing system to: detect a creation of a first thread; activate thread based allocation for the first thread based on the first thread being associated with user interface activity; switch to a normal allocation mode from the thread based allocation for a third thread based on the third thread not being associated with the user interface activity; dedicate a first memory region to objects associated with only the first thread to bypass a storage of objects associated with a second thread into the first memory region based on the thread based allocation being activated for the first thread; dedicate a second memory region to the objects associated with the second thread; execute the normal allocation mode for objects associated with the third thread to store the objects of the third thread in a shared space based on the third thread being switched to the normal allocation mode; and conduct a first reclamation of the first memory region in response to a termination of the first thread.

14. The at least one non-transitory computer readable storage medium of claim 13, wherein the first memory region is a heap region and the first reclamation is to bypass a pause phase and a copy phase of a garbage collection process with respect to the heap region.

15. The at least one non-transitory computer readable storage medium of claim 13, wherein to dedicate the first memory region to the objects associated with the first thread, the executable program instructions, when executed, cause the computing system to: map the first thread to the first memory region.

16. The at least one non-transitory computer readable storage medium of claim 15, wherein the executable program instructions, when executed, cause the computing system to: deactivate the thread based allocation in response to the termination of the first thread; and unmap the first thread from the first memory region.

17. The at least one non-transitory computer readable storage medium of claim 13, wherein the executable program instructions, when executed, cause the computing system to: dedicate a third

memory region to the objects associated with the first thread in response to a determination that the first memory region is full; and conduct a second reclamation of the third memory region in response to the termination of the first thread.

18. The at least one non-transitory computer readable storage medium of claim 13, wherein the first and second threads are to correspond to the user interface activity.

19. A method comprising: detecting a creation of a first thread; activating thread based allocation for the first thread based on the first thread being associated with user interface activity; switching to a normal allocation mode from the thread based allocation for a third thread based on the third thread not being associated with the user interface activity; dedicating a first memory region to objects associated only with the first thread to bypass a storage of objects associated with a second thread into the first memory region based on the thread based allocation being activated for the first thread; dedicating a second memory region to the objects associated with the second thread; executing the normal allocation mode for objects associated with the third thread to store the objects of the third thread in a shared space based on the third thread being switched to the normal allocation mode; and conducting a first reclamation of the first memory region in response to a termination of the first thread.

20. The method of claim 19, wherein the first memory region is a heap region and the first reclamation bypasses a pause phase and a copy phase of a garbage collection process with respect to the heap region.

21. The method of claim 19, wherein dedicating the first memory region to the objects associated with the first thread includes: mapping the first thread to the first memory region.

22. The method of claim 21, further including: deactivating the thread based allocation in response to the termination of the first thread; and unmapping the first thread from the first memory region.

23. The method of claim 19, further including: dedicating a third memory region to the objects associated with the first thread in response to a determination that the first memory region is full; and conducting a second reclamation of the third memory region in response to the termination of the first thread.

24. The method of claim 19, wherein the first and second threads correspond to the user interface activity.